Login-System implementieren und analysieren

Allgemeine Informationen

Hilfsmittel: Eigenes Notebook
Nachweis: Aufgaben abgeschlossen, Notizen/Bemerkungen zur Umsetzung notiert
Sozialform: Einzelarbeit

Ausgangslage

Du arbeitest als Applikationsentwickler/in in einem Softwareunternehmen. Dein Team entwickelt eine neue Webplattform für einen Onlinekursanbieter.

Auf der Plattform können Benutzer:

- sich registrieren und anmelden
- ihre persönlichen Kursdaten ansehen
- Kurse buchen oder absolvieren

Es gibt dabei unterschiedliche Benutzerrollen: Administratoren, Studenten und Gäste

Damit die Plattform sicher ist, muss ein zuverlässiges Authentifizierungssystem implementiert werden

Benötigte Tools

Es wird eine IDE benötigt (z. B. Visual Studio Code), ein Browser und Postman (für die Analyse und das Senden von Requests).

Aufgabe 1: Vorbereitung und Installation

Arbeitsauftrag

Erstelle in einer IDE dein Projekt und führe folgende Befehle im Terminal aus:

```
npm init -y
npm install express express-session jsonwebtoken bcrypt
```

Füge den Code aus der Vorlage in das Projekt ein. Starte das Projekt mit `node server.js` (dieser Befehl ist auch durchzuführen, wenn Änderungen am Code erfolgt sind). Das Projekt ist nun unter http://localhost:3000 erreichbar.

Aufgabe 1: Kennwort sicher speichern

Arbeitsauftrag

Die Benutzerdaten sind in einer externen JSON-Datei abgelegt. Die Kennwörter dürften künftig nicht mehr als Klartext gespeichert werden.

- Erstelle aus der JSON-Datei mit Klartext-Kennwörtern eine neue users.json-Datei, aber mit bcrypt-Hashes der bereits verwendeten Kennwörter (bcrypt.hash() mit Salt).
- Passe das Login so an, dass bei der Anmeldung bcrypt.compare() verwendet wird.
- Teste das Login mit einem gültigen und einem ungültigen Kennwort mit Postman.

Recherchiere bei Unklarheiten die Handhabung von bcrypt selbstständig.

Installation/Einrichtung

Paket in der server.js-Datei importieren

```
const bcrypt = require("bcrypt");
```

Aufgabe 2: Sessionbasierte Authentifizierung einbinden

Arbeitsauftrag

Der Loginprozess soll so angepasst werden, dass nach der erfolgreichen Anmeldung eine Session erstellt wird. Eine neue Seite mit dem Benutzerprofil soll nur aufgerufen werden können, wenn eine Session besteht.

- Nach erfolgreichem Login soll eine Session erstellt werden.
- Der Benutzername und die Rolle sollen in der Session gespeichert werden.
- Einrichtung einer neuen, geschützte Route (GET «/profile»), die nur mit einer existierenden, gültigen Session funktioniert.
- Achte bei der Antwort (ob gültig oder nicht) darauf, den richtigen http-Statuscode zu verwenden.

Recherchiere bei Unklarheiten die Handhabung Sessions mit «express-session» selbstständig.

Installation/Einrichtung

Paket «express-session»

```
npm install express-session
```

Paket in der server.js-Datei importieren

```
const session = require("express-session");
```

Session-Middleware in der server.js-Datei hinzufügen

```
app.use(session({
  secret: "bbzw",
  resave: false,
  saveUninitialized: false,
  cookie: {
    httpOnly: true,
    secure: false,
    sameSite: "lax"
  }
}));
```

Aufgabe 3: Abmeldung implementieren

Arbeitsauftrag

Es soll nun noch eine Funktionalität zur Abmeldung der User erstellt werden. Bei der Abmeldung müssen die Cookies gelöscht werden.

- Einrichtung einer neuen, geschützte Route (POST «/logout»).
- Achte bei der Antwort (ob gültig oder nicht) darauf, den richtigen http-Statuscode zu verwenden.
- Nach erfolgreichem Logout soll die Session gelöscht werden.

Überprüfung

- Führe die Anmeldung mit Postman durch («/login»).
- Rufe die Profilseite mit Postman auf («/profile»). Falls du angemeldet bis, wird dein Profil angezeigt.
- Führe die Abmeldung mit Postman durch («/logout»).
- Rufe nun nochmals die Profilseite mit Postman auf («/profile»). Du solltest nicht eingeloggt sein.

Aufgabe 4: Tokenbasierte Authentifizierung einbinden

Aufgabenbeschreibung

Nun soll die bestehende Lösung erweitert werden, so dass JWT Tokens verwendet werden. Die sessionbasierte Authentifizierung wird nicht mehr benötigt und kann entfernt werden. Lege dafür am besten eine Kopie der bestehenden JavaScript-Datei an.

Arbeitsauftrag

- Nach erfolgreichem Login soll ein JWT Token erstellt werden.
- Das Token enthält Informationen über den angemeldeten User (Username und Rolle).
- Achte bei der Antwort (ob gültig oder nicht) darauf, den richtigen http-Statuscode zu verwenden.

Installation

Paket in der server.js-Datei importieren

```
const jwt = require("jsonwebtoken");
```

Token erstellen (ist bei der Login-Route hinzuzufügen)

```
const token = jwt.sign(
  {
    user: username
  },
  "bbzw",
  {
```

```
    expiresIn: "5m"
  }
);

res.json({token});
```

Überprüfung

- Führe die Anmeldung mit gültigen Zugangsdaten mit Postman durch («/login»). Du solltest nun als Response ein Token erhalten.
- Führe die Anmeldung mit ungültigen Zugangsdaten mit Postman durch («/login»). Du solltest nun eine Response mit entsprechendem http-Statuscode erhalten.

Aufgabe 5: Tokenbasierte Authentifizierung für API nutzen

Aufgabenbeschreibung

Es soll nun ein neuer API-Endpunkt generiert werden. Dieser Endpunkt soll nur aufgerufen werden können, wenn eine gültiger JWT Token besteht.

- Einrichtung einer neuen, geschützte Route (POST «/api»), die nur mit einem existierenden, gültigen JWT Token funktioniert.
- Achte bei der Antwort (ob gültig oder nicht) darauf, den richtigen http-Statuscode zu verwenden.

Überprüfung

- Führe die Anmeldung mit Postman durch («/login»).
- Rufe die API mit Postman auf («/api»). Falls du angemeldet bist, wird eine entsprechende Meldung angezeigt.

Aufgabe 6: Account Lockout implementieren

Aufgabenbeschreibung

Füge eine zusätzliche Sicherheitsmassnahme ein, indem du bei mehr als 5 Fehlversuchen einer Anmeldung den entsprechenden User sperrst. Diese Information (ob ein User gesperrt ist oder nicht) muss nicht in der JSON-Datei abgelegt werden.

Überprüfung

- Führe die Anmeldung fünf Mal mit ungültigen Zugangsdaten mit Postman durch («/login»).
- Beim 6. Versuch soll eine entsprechende Meldung angezeigt werden.

Aufgabe 7: Userregistration implementieren

Aufgabenbeschreibung

Es soll nun möglich sein, dass sich User registrieren können. User sollen einen Username definieren und ein Kennwort definieren können. Das Kennwort muss verschiedenen Richtlinien entsprechen.

- Einrichtung einer neuen, geschützte Route (POST «/register»).
- Der Username soll eindeutig (unique) sein.
- Das gewählte Kennwort muss folgenden Kriterien entsprechen: Mindestens 10 Zeichen lang, mindestens 1 grossgeschriebener Buchstabe, mindestens 1 kleingeschriebener Buchstabe, mindestens 1 Sonderzeichen («*», «+», «%», «&», «?», «$»), mindestens 1 Zahl
- Der neue User muss nicht effektiv angelegt werden (in der JSON-Datei), es reicht, wenn eine entsprechende Meldung angezeigt wird, wenn man sich registriert.

Überprüfung

- Führe eine Registration mit ungenügenden Kennwortrichtlinien durch («/register»). Du solltest eine Meldung erhalten, dass das Kennwort nicht den Richtlinien entspricht.
- Führe eine Registration mit genügenden Kennwortrichtlinien durch («/register»). Du solltest eine Meldung erhalten, dass der User erfolgreich angelegt wurde.
- Meldungen wie bei den beiden obenstehenden Punkten sollen auch bei einem bereits angelegten User (also verwendeten Username) gemacht werden.
- Nach einer erfolgreichen Registration soll man sich anmelden können.